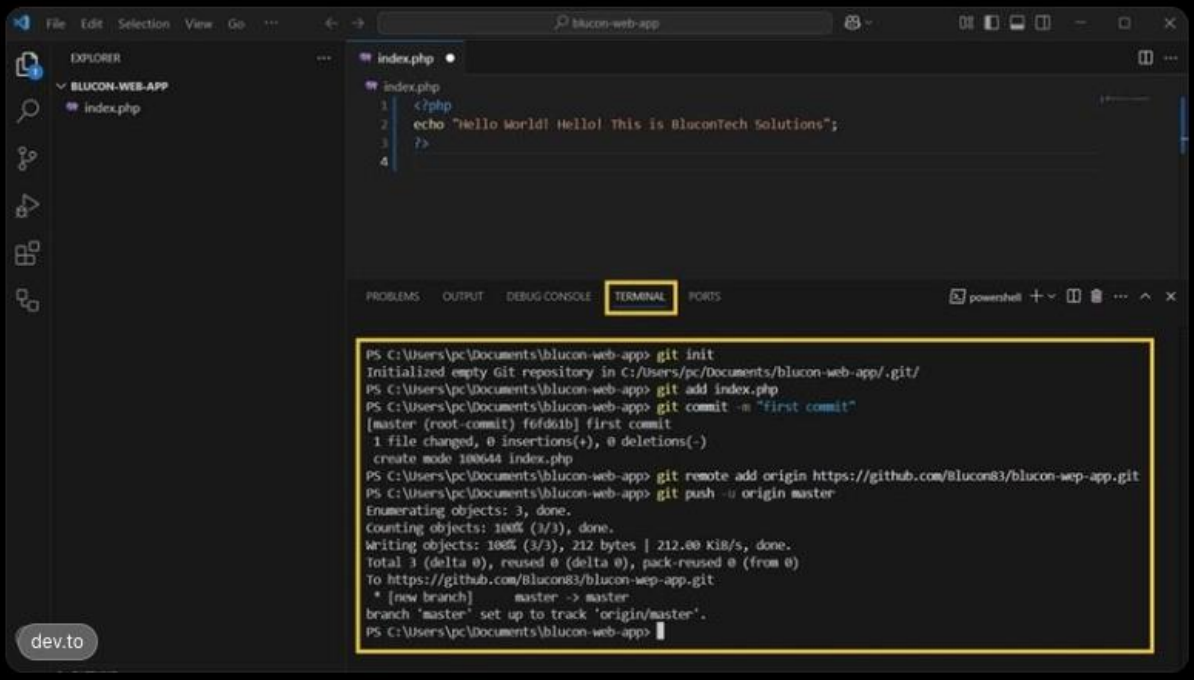
Real-time CI/CD pipeline — step by step

1. Developer pushes new code to GitHub



What happens?

The developer changes the application code on their laptop. For example, they fix a bug in the checkout API.

They save the changes, commit them using Git, and push them to the GitHub repository.

Example commands:

```bash
git add .
git commit -m "Fixed checkout API issue"
git push origin main
```

In simple words: The developer has completed the code change and uploaded the latest version of the code to GitHub so the team can access it.

3. Jenkins checks out the code and builds the application

What happens?

Jenkins starts a job on a Jenkins agent, which is the machine that performs the pipeline tasks.

First, Jenkins downloads the latest code from GitHub into its workspace. This process is called *checkout*.

Next, Jenkins builds the application. For example, if it is a Java application, Jenkins might run:

```bash
mvn clean package
```

Maven downloads required dependencies, compiles the Java code, and packages the application.

In simple words: Jenkins takes the latest code from GitHub and checks whether it can successfully build the application.

If the build fails, the pipeline usually stops, and Jenkins reports the failure.

4. Automated tests, code-quality checks, and security scans run

What happens?

After a successful build, Jenkins checks whether the code works correctly and meets the project's quality and security requirements.

Automated testing — JUnit or pytest

Checks whether the application functions correctly. For example, does the checkout API calculate the order total correctly?

Code-quality analysis — SonarQube

Checks the code for bugs, code-quality problems, and certain security issues.

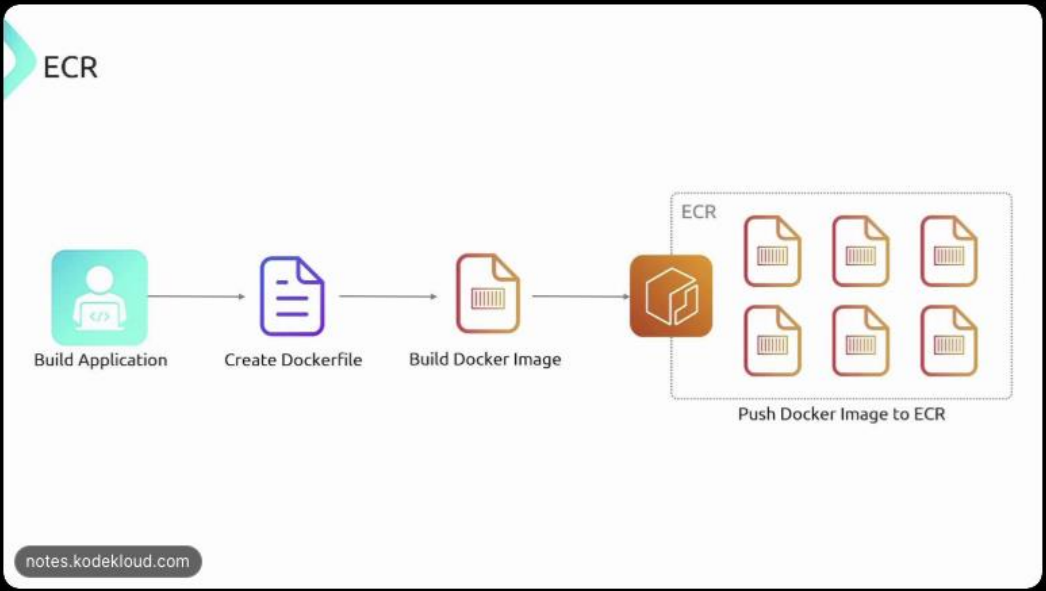
Security scanning — Trivy or Snyk

Checks for known vulnerabilities in dependencies and, when configured, container images.

In simple words: Before releasing the new code, Jenkins checks whether it builds correctly, works as expected, and meets the team's quality and security rules.

If a required test fails, Jenkins normally stops the pipeline so the faulty code does not proceed to deployment.

5. Jenkins creates a Docker image and pushes it to Amazon ECR



This stage has two main parts.

Part A: Build the Docker image

After the required checks pass, Jenkins uses a `Dockerfile` to package the application and its runtime requirements into a Docker image.

For example:

```bash
docker build -t checkout-app:v12 .
```

Here, `v12` is an example version tag. The actual versioning method depends on the project.

Think of the Docker image as a package containing what is needed to run the application.

Part B: Push the image to Amazon ECR

Amazon ECR is a service that stores Docker container images.

Jenkins authenticates to ECR and uploads the image:

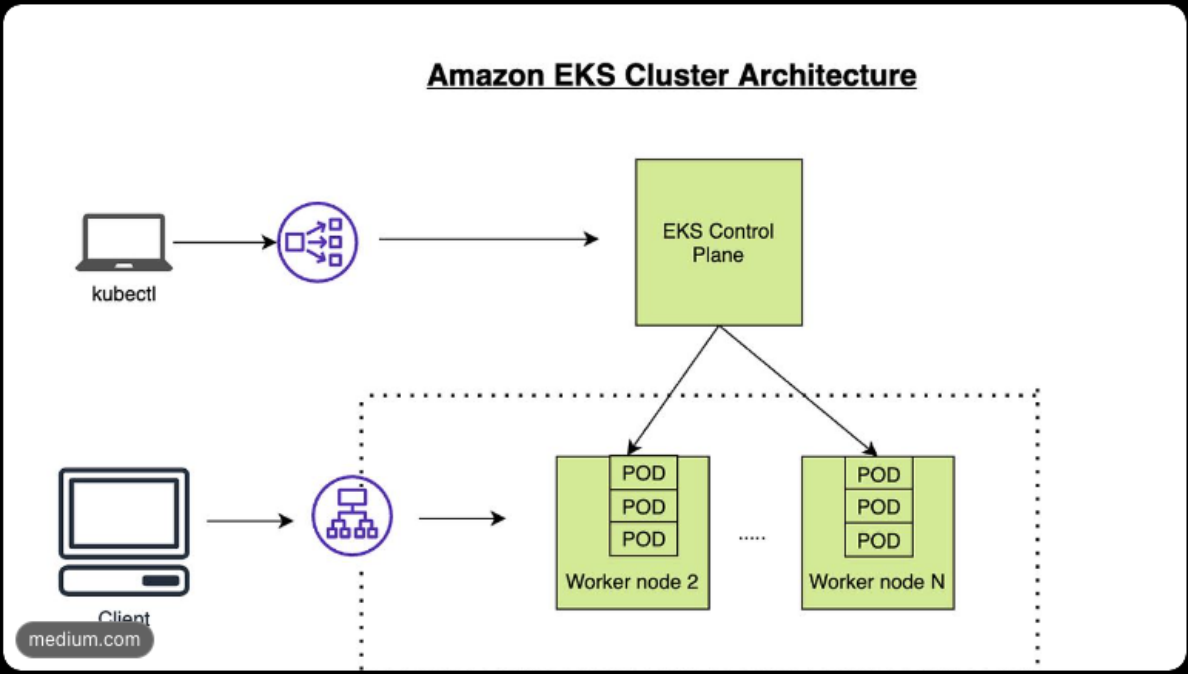
```bash
docker push <ecr-repository>/checkout-app:v12
```

The repository address is a placeholder; the actual address comes from your AWS account and ECR configuration.

In simple words: Jenkins packs the new application into a Docker image and stores it in ECR. Kubernetes can then download that exact image when deploying the application.

An important point: ECR stores the image; it does not run the application.

6. The deployment system updates the image version in Kubernetes on Amazon EKS



What happens?

Amazon EKS is AWS's managed Kubernetes service. Kubernetes manages the application containers running in the cluster.

The deployment process tells Kubernetes to use the new image, such as `checkout-app:v12`, instead of the old image, `checkout-app:v11`.

For example, the deployment might run this command:

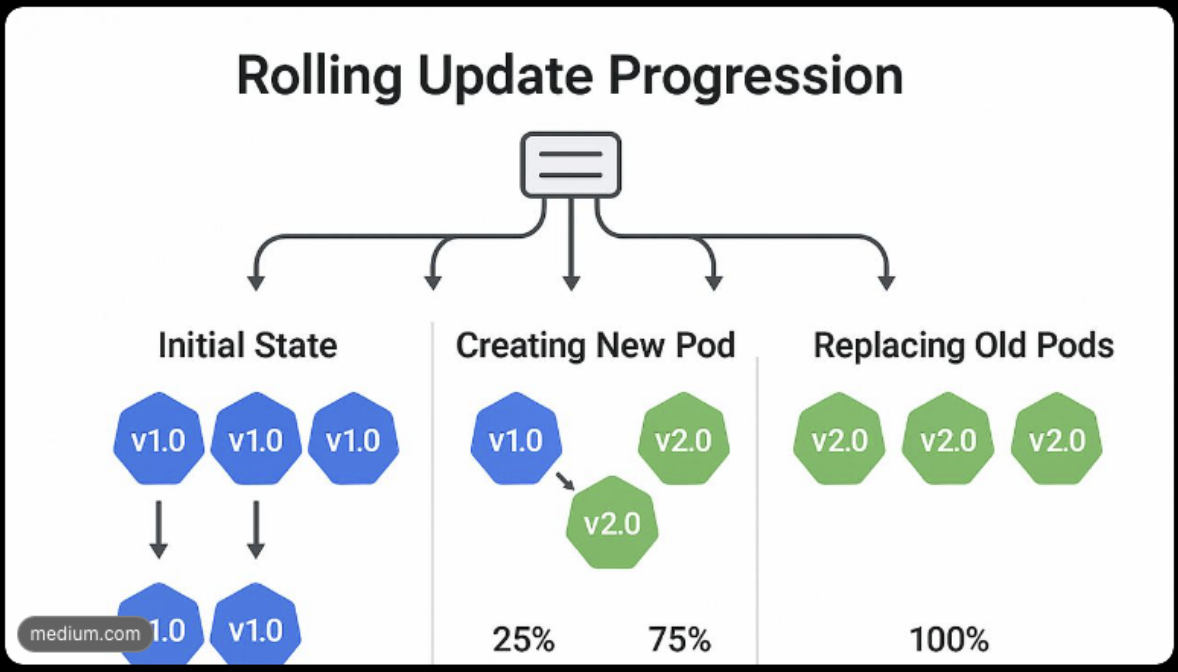
```bash
kubectl set image deployment/checkout-api \
  checkout-api=<ecr-repository>/checkout-app:v12
```

The names and image address are examples. They must match the actual Kubernetes Deployment, container, and ECR repository.

The deployment system could be Jenkins running `kubectl`, Helm, or a GitOps tool such as Argo CD.

In simple words: Kubernetes is told, "Use the new version of the checkout application instead of the old version."

7. Kubernetes performs a rolling update



What happens?

Suppose the old checkout application is running in three Pods, all using image `v11`.

Kubernetes gradually starts new Pods using image `v12` and replaces the old Pods as the new ones become ready, according to the Deployment's rollout settings.

Before deployment

Pod 1	Pod 2	Pod 3
v11	v11	v11

↓↑ Rolling update

After deployment

Pod 1	Pod 2	Pod 3
v12	v12	v12

Illustration: the old Pods are replaced gradually. The exact number of Pods available during the update depends on the rollout settings and cluster capacity.

In simple words: Kubernetes does not necessarily stop all the old application Pods at once. It replaces them gradually, helping the application continue serving customers during deployment.

A rolling update does not guarantee zero downtime by itself. Readiness probes, sufficient capacity, correct rollout settings, and application compatibility also matter.

8. Automated health checks and smoke tests validate the checkout API

What happens?

After deployment, the team verifies that the new version is actually working.

There are two important checks:

- **Health checks:** Kubernetes checks whether the application is running and ready to receive traffic. For example, a readiness probe checks whether the checkout service is ready.
- **Smoke tests:** An automated test sends a basic request to the checkout API to verify that the main functionality works.

For example:

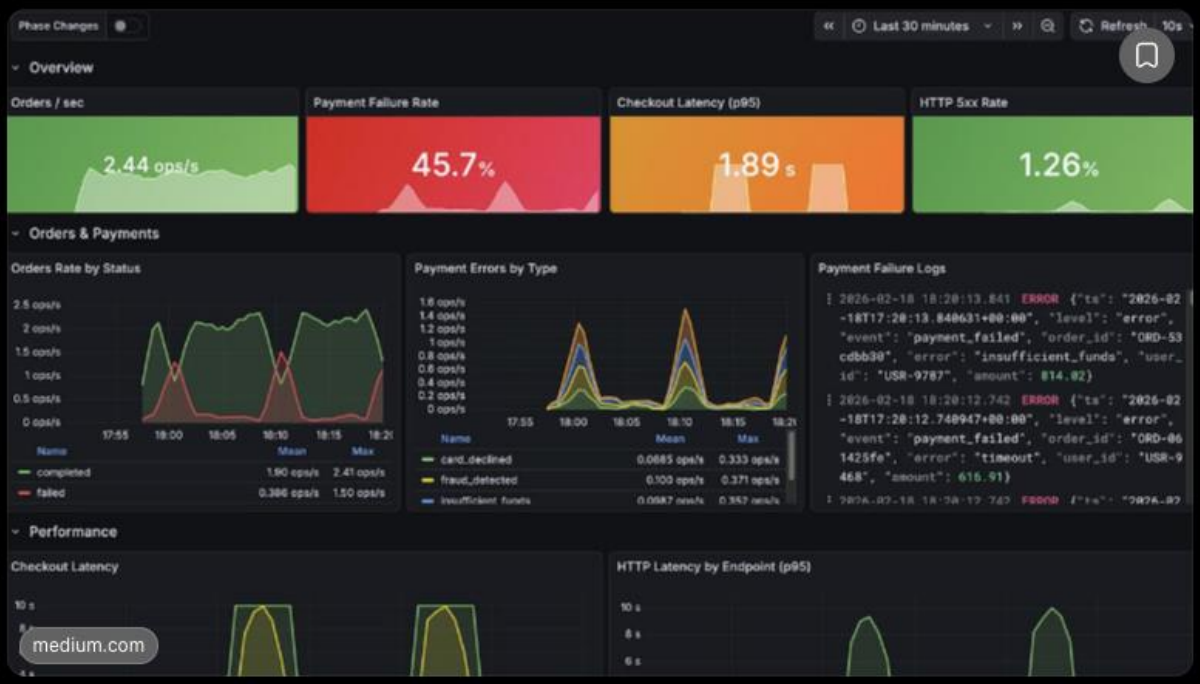
```bash
curl -f https://example.com/health
```

This is an illustrative health endpoint. The application must implement that endpoint, and the actual URL must be configured.

A successful health response is useful, but a separate smoke test may also need to verify that checkout works correctly.

In simple words: We check whether the new application is alive, ready, and able to handle important requests. If the checks fail, the release may be stopped or rolled back according to the pipeline's rules.

9. Prometheus, Grafana, CloudWatch, and log tools monitor the application



What happens?

Even if the deployment succeeds, the new version might have a problem that appears only when real customers use it.

The monitoring tools help the DevOps team identify those problems.

Tool	What it does
Prometheus	Collects and stores metrics, such as request counts and error rates.
Grafana	Displays metrics in dashboards and helps visualize performance.
AWS CloudWatch	Collects configured AWS metrics, logs, and alarms.
ELK or Loki	Helps collect, search, and analyze application logs.

For example, after the checkout update, the team may notice that API response times have increased or error rates have gone up.

They can use metrics and logs to investigate what is happening.

In simple words: Monitoring helps the team check whether the application continues working properly after deployment, not just whether the deployment itself succeeded.

10. If the new release causes errors, the team rolls back

What happens?

Imagine version `v12` was deployed successfully, but customers start getting errors when placing orders.

The team investigates the issue using application logs, metrics, and alerts. If the new release is identified as the cause, they can restore the previous stable version, `v11`.

For a Kubernetes Deployment, a simple rollback command is:

```bash
kubectl rollout undo deployment/checkout-api
```

This rolls back to the previous recorded Deployment revision, assuming the revision is available and the deployment is managed in a compatible way.

In a GitOps setup, the team may instead revert the change in Git and let the GitOps controller reconcile the cluster.

In simple words: If the new version breaks the application, the team returns to the previous working version to restore service while investigating and fixing the problem.

Rollback is not always enough by itself. Database changes, data migrations, and external side effects may require a separate recovery plan.

The complete process in one simple example

Imagine you are the DevOps Engineer responsible for this application:

Developer

Fixes the checkout bug and pushes the code to GitHub.

Jenkins

Downloads the code, builds the application, runs tests, and performs configured quality and security checks.

Docker and ECR

Jenkins creates a versioned image and stores it in the registry.

Kubernetes on EKS

Deploys the new image and gradually replaces old Pods.

DevOps and monitoring

Verify the new version, monitor its performance, investigate errors, and roll back if necessary.

2. AWS example — where Terraform sits

Let's say your e-commerce application runs on AWS using Jenkins, Docker, Amazon ECR, and Amazon EKS.



What exactly does Terraform create on AWS?

AWS resource	Why we need it
VPC and subnets	Provide the application's network environment.
IAM roles and policies	Control which services and users can access AWS resources.
EKS cluster and node groups	Provide the Kubernetes environment for the application.
ECR repository	Store Docker images.
Load balancer and related networking	Route external or internal traffic to the application.
RDS database, if required	Store application data.

For example, Terraform might create an EKS cluster once. Later, a developer changes the checkout API, and Jenkins deploys a new image to that existing cluster. **Terraform does not need to recreate the EKS cluster for every application release.**

What happens in the AWS Terraform pipeline?

1. You write Terraform configuration files, such as `main.tf`, `variables.tf`, and `outputs.tf`.
2. You push the changes to GitHub.
3. Jenkins checks out the Terraform code.
4. Jenkins runs `terraform init` to initialize Terraform and its providers.
5. It runs `terraform validate` and `terraform plan` to check the configuration and review proposed infrastructure changes.
6. A reviewer approves the plan when required by the team's process.
7. Jenkins runs `terraform apply` to create or update the approved resources.
8. The infrastructure becomes available for the application deployment pipeline.

Example commands:

```bash
terraform init
terraform validate
terraform plan
terraform apply
```

In production, the apply step should use a reviewed plan and appropriate approval controls. Terraform state should be stored securely, usually in a shared remote backend such as an S3 bucket with state locking configured appropriately.

“In my project, we used Terraform to manage our cloud infrastructure. We maintained the Terraform configuration files in GitHub, and when infrastructure changes were required, we ran a separate pipeline to validate the configuration, review the plan, and apply the changes.

For AWS, Terraform managed resources such as VPCs, IAM roles, EKS clusters, and other required services. For GCP, it managed resources such as VPCs, IAM permissions, GKE clusters, and other cloud services.

Once the infrastructure was ready, our application CI/CD pipeline handled the application code. It built and tested the code, created a Docker image, pushed it to the container registry, and deployed it to Kubernetes.

We did not need to recreate the infrastructure every time a developer changed the application code. We used Terraform whenever the infrastructure needed to be created or updated.”